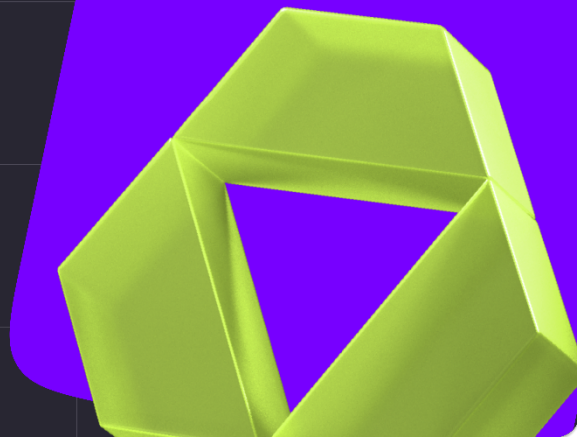


Постридер

SQL для начинающих

Занятие 11



Теперь вы знаете, что такое представления (views): как их создавать, обновлять и удалять. **VIEW** — это сохранённый SQL-запрос, который ведёт себя как таблица, но не хранит данные физически.

● Представления (views) — ключевые понятия

Представление (view) — виртуальная таблица, результат сохранённого **SELECT**. Данные не дублируются: при каждом обращении к представлению запрос выполняется заново на актуальных данных.

Три главных преимущества views:

Упрощение сложных запросов — пишем **SELECT * FROM** active_users вместо повторяющегося запроса с **WHERE**.

Безопасность данных — можно показывать только нужные столбцы, скрывая чувствительную информацию (пароли, телефоны).

Логическая структуризация — представление отражает бизнес-логику (например, "новые пользователи" или "VIP-клиенты").

● Задача. Представление для активных пользователей

Создать базовое представление, отфильтровав только активных пользователей. После создания к представлению обращаются как к обычной таблице.

```
CREATE VIEW active_users AS
SELECT id, name, email
FROM users
WHERE status = 'active';
```

Теперь можно писать просто **SELECT * FROM** active_users — вместо того чтобы каждый раз вспоминать условие **WHERE** status = 'active'.

Важно: представление — не копия данных. Если добавить нового активного пользователя в таблицу users, он сразу появится в active_users без каких-либо дополнительных действий.

● Задача. Динамическое представление с NOW() и INTERVAL

Создать представление для пользователей, зарегистрировавшихся за последние 30 дней.

Ключевая особенность: `NOW()` вычисляется каждый раз при обращении к представлению, а не при его создании.

```
CREATE VIEW new_users_view AS
SELECT id, name, email, created_at
FROM users
WHERE created_at >= NOW() - INTERVAL '30 days';
```

Это означает, что представление автоматически «скользит»: сегодня оно показывает данные с 31 января, завтра — с 1 февраля. Нет нужды обновлять представление вручную.

● Задача. Маскировка данных через строковые функции

Создать представление, которое показывает только первые 3 символа email, заменяя остальные звёздочками. Это пример защиты персональных данных на уровне базы.

```
CREATE VIEW masked_email_view AS
SELECT
    id,
    name,
    CONCAT(SUBSTRING(email, 1, 3),
           REPEAT('*', LENGTH(email) - 3)) AS masked_email
FROM users;
```

Разбор функций:

`SUBSTRING(email, 1, 3)` — берёт символы с позиции 1, длиной 3. Результат: первые три буквы.

`LENGTH(email) - 3` — сколько символов нужно заменить звёздочками (общая длина минус первые три).

`REPEAT('*', n)` — создаёт строку из n звёздочек.

`CONCAT(...)` — склеивает первые символы и звёздочки в одну строку.

Результат: 'ivan.ivanov@example.com' → 'iva*****'

● Задача. Просмотр кода представления

Посмотреть какой SQL-запрос скрыт внутри представления. Это полезно, когда нужно проверить логику или создать похожее представление.

```
SELECT definition
FROM pg_views
WHERE viewname = 'masked_email_view';
```

Или через специальную функцию (второй параметр true включает красивое форматирование):

```
SELECT pg_get_viewdef('masked_email_view', true);
```

`pg_views` — системный каталог PostgreSQL, в котором хранятся определения всех представлений текущей базы данных.

Нюансы

● Обновляемые и необновляемые представления

Не все представления можно использовать с **INSERT**, **UPDATE** или **DELETE**. Простые представления (одна таблица, без агрегаций) — обновляемые. Сложные — нет.

Представление обновляемо, если:

- ссылается на одну таблицу,
- не содержит **DISTINCT**, **GROUP BY**, **HAVING**, **LIMIT**, **OFFSET**, агрегатных функций,
- все столбцы напрямую соответствуют столбцам базовой таблицы.

-- Это обновляемое представление:

```
UPDATE active_users
SET name = 'Иван Иванов'
WHERE id = 1; -- сработает
```

● Материализованные представления хранят данные на диске

Обычный **VIEW** выполняет запрос каждый раз при обращении. **MATERIALIZED VIEW** сохраняет результат физически — это ускоряет доступ, но данные могут устаревать.

```
CREATE MATERIALIZED VIEW mat_view_name AS
SELECT columns
FROM tables
WHERE conditions
WITH [NO] DATA;
```

WITH DATA (по умолчанию) — немедленно выполняет запрос и сохраняет данные при создании.

WITH NO DATA — создаёт пустое представление, данные заполнятся при первом **REFRESH**.

Данные в материализованном представлении не обновляются автоматически — нужно вызвать **REFRESH** вручную:

```
REFRESH MATERIALIZED VIEW mat_view_name;
DROP MATERIALIZED VIEW [IF EXISTS] mat_view_name [CASCADE | RESTRICT];
```

● Права доступа к представлениям

Можно давать пользователям права на представление, не открывая доступ к исходным таблицам. Это классический паттерн безопасности: аналитик видит только те данные, которые ему нужны.

```
GRANT SELECT, INSERT, UPDATE, DELETE ON view_name TO user_name;
```

Например: аналитику дать **GRANT SELECT ON** `masked_email_view` — он увидит только замаскированные email, но не сможет обратиться к таблице `users` напрямую.

● OR REPLACE и DROP VIEW: управление жизненным циклом

Представление можно обновить двумя способами: пересоздать командой **CREATE OR REPLACE VIEW** или удалить старое через **DROP VIEW** и создать новое.

CREATE OR REPLACE VIEW — заменяет определение без предупреждения. Удобно при обновлениях, но используйте осторожно: ошибка перезапишет рабочее представление.

DROP VIEW IF EXISTS `active_users;` — удаляет только представление, данные в таблицах остаются нетронутыми.